Assignment 3E, 2025

DD2434 Machine Learning, Advanced Course

Bruno Carchia carchia@kth.se

Riccardo Alfonso Cerrone cerrone@kth.se

January 5, 2026

Group # 60

3E.1 Principal Component Analysis

Question 3.1.1

We are assuming that the columns of **W** are orthonormal ($W^TW = I_{k\times k}$): in other words W is full rank and its columns are a basis of the space that it spans. However WW^T is not necessarily equal to $I_{d\times d}$.
We are assuming that because in PCA we want to find a lower dimensional representation and **W** represents the principal directions/components onto which we project our data which should be uncorrelated. This ensures that each principal component captures distinct , non redundant information about the data's variability

Question 3.1.2

We want to derive $x = W^+y$ from $y = Wx$ knowing that $W^+ = V\Sigma^+U^T$
We can take the basic equation and multiply it by W^T

$$W^Ty = W^TWx$$

In PCA we assume that W is a matrix with orthonormal columns , so $W^TW = I_{k\times k}$

$$W^Ty = x$$

We know, from the Module 7 slides, that if W has linearly independent columns then it can be rewritten as

$$W^+ = (W^TW)^{-1}W^T = (I_{k\times k})^{-1}W^T = W^T$$

We can then replace it into the intermediate equation to get the final form.

$$W^+ = W^T$$

$$x = W^+y$$

Question 3.1.3

It is possible to apply this formula because W in our assumption has linearly indepedent columns.

$$W^+ = (W^T W)^{-1} W^T = (I_{k\times k})^{-1} W^T = W^T$$

Question 3.1.4

If we perform PCA on data that has not been centered, you will encounter the following issue: First Principal Component Bias, where the first principal component will often point from the origin toward the center of the data cluster rather than along the direction of maximum variance. This happens because the model tries to minimize the squared distance to the origin, treating the mean offset as a massive "signal" to be captured.

Question 3.1.5

Answer: It depends on the centering requirements.

A single SVD operation can provide both row and column structure, but **proper PCA on both rows and columns typically requires different preprocessing** (centering). We demonstrate the mathematical relationship below. Problem feature:

- $Y \in \mathbb{R}^{n\times d}$ (data matrix, assumed appropriately centered)
- $X \in \mathbb{R}^{n\times k}$ (encoded representation)
- $W \in \mathbb{R}^{d\times k}$ (projection matrix with orthonormal columns)
- n is the number of datapoints
- d is the number of dimensions
- k is the desired number of dimensions ($k \leq d$)

We are inverting the rows and the columns w.r.t. the common representation of Y and X which is $Y \in \mathbb{R}^{d\times n}$, $X \in \mathbb{R}^{k\times n}$.

The mapping functions are:

- Encoding: $X = YW$
- Decoding: $\hat{Y} = XW^T = YWW^T$

The MSE can be seen as a reconstruction error:

$$\begin{aligned}
\mathbb{E}_W &= \mathbb{E}_Y \left[\|Y - \text{dec}(\text{enc}(Y))\|_2^2\right] \\
&= \mathbb{E}_Y \left[\|Y - YWW^T\|_2^2\right] \\
&= \mathbb{E}_Y \left[(Y - YWW^T)^T (Y - YWW^T)\right] \\
&= \mathbb{E}_Y \left[Y^T Y - Y^T YWW^T - WW^T Y^T Y + WW^T Y^T YWW^T\right] \\
&= \mathbb{E}_Y [Y^T Y] - \text{Tr}(Y^T YWW^T) - \text{Tr}(WW^T Y^T Y) + \text{Tr}(WW^T Y^T YWW^T)
\end{aligned}$$

As in the slides of Module 7, $\mathbb{E}_Y[Y^TY]$ is constant and can be ignored for the optimization purpose.

Using the cyclic property of the trace, where $\text{Tr}(Y^TYWW^T) = \text{Tr}(WW^TY^TY)$:

$$\mathbb{E}_W \approx -2\text{Tr}(WW^TY^TY) + \text{Tr}(WW^TY^TYWW^T)$$

Since $W^TW = I$ (orthonormal columns), we have:

$$\text{Tr}(WW^TY^TYWW^T) = \text{Tr}(Y^TYWW^T) = \text{Tr}(WW^TY^TY)$$

Therefore:

$$\mathbb{E}_W \approx -\text{Tr}(WW^TY^TY)$$

To minimize $\mathbb{E}_W$, we maximize $\text{Tr}(WW^TY^TY)$.

We apply the SVD on Y as $Y = U\Sigma V^T$ where:

$$U \in \mathbb{R}^{n\times n}, \quad \Sigma \in \mathbb{R}^{n\times d}, \quad V \in \mathbb{R}^{d\times d}$$

with U and V having orthonormal columns.

For the optimization, we consider the top k components:

$$\begin{aligned}
\text{Tr}\left(WW^T(U\Sigma V^T)^T(U\Sigma V^T)\right) &= \text{Tr}\left(WW^TV\Sigma^TU^TU\Sigma V^T\right) \\
&= \text{Tr}\left(WW^TV\Sigma^2V^T\right) \quad (\text{since } U^TU = I) \\
&= \text{Tr}\left(V^TWW^TV\Sigma^2\right) \quad (\text{cyclic property})
\end{aligned}$$

The optimal W is given by the first k columns of V, which gives:

$$\text{Tr}(\Sigma_k^2) = \sigma_1^2 + \sigma_2^2 + \ldots + \sigma_k^2$$

where $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_k$ are the top k singular values.

Conclusion: The Centering Caveat

Mathematical insight: A single SVD of Y provides both:

- U: Left singular vectors (related to row structure)
- V: Right singular vectors (related to column structure)

However, for proper PCA:

1. **PCA on rows** (treating rows as datapoints):
   - Requires: Column-centered Y (subtract mean of each column)
   - Uses: Right singular vectors V
2. **PCA on columns** (treating columns as datapoints):
   - Requires: Row-centered Y (subtract mean of each row)

- Uses: Left singular vectors U

Final answer:

- A single SVD gives you both U and V, capturing row and column structure
- **But** proper PCA on both rows and columns generally requires performing SVD on two differently centered versions of the data
- Exception: If the data has special structure (e.g., already centered both ways, or if centering is not required for the application), then one SVD suffices

The singular values remain the same under transposition, but the centering differs, which is why we typically need separate preprocessing for row-PCA vs. column-PCA.

Question 3.1.6

When we work with PCA, we should normalize each column (feature) when attributes represent quantities in different units but it should be avoided when values between attribues are comparable.
The SVD of Y is

$$Y = U\Sigma V^T$$

We can replace it in the Y' definition

$$Y' = DY = DU\Sigma V^T$$

The matrix $U' = DU$ is not necessarily orthonormal.

$$U'U'^T = (DU)^T(DU) = U^T D^T DU = U^T D^2 U$$

In order to be orthonormal $D^2 = I$, it happens only when $c_j = \pm 1$
It means that the left singular vectors are not simply scaled versions of those of Y and that Y and Y' have not the same singular values. The SVDs are related but not identical. The right singular vectors V might remain the same in special cases, but generally both U and Σ will change when we scale the features.
PCA wants to find the directions with the highest variance by minimizing the trace (showed above) and the optimal solution is represented by

$$W = U_k$$

Where U_k is obtained from the SVD of $Y = U\Sigma V^T$ by taking only the k columns of U that correspond to the k largest singular values.
We have proved that the SVDs of Y and Y' are different , for this reason the solutions to the optimal problem are different: the PCA directions for Y' are not the same as for Y. In conclusion, PCA is sensitive to feature scaling.

3E.2 Isomap

Question 3E.2.1

As discussed during the lectures, in Isomap the neighborhood graph G is constructed by connecting each data point to its ρ-nearest neighbors. This construction, however, does not guarantee that the resulting graph is connected.

The graph G may become disconnected when the chosen neighborhood size ρ is too small relative to the data distribution. In such cases, points that are close along the underlying manifold may fail to be connected, either directly or indirectly, in the neighborhood graph.

As a simple illustrative example, we consider the following set of points embedded in two dimensions.

Note Although the example is illustrated in two dimensions for visualization purposes, the ρ-nearest neighbor graph construction is independent of the ambient dimension and applies identically in higher-dimensional spaces.

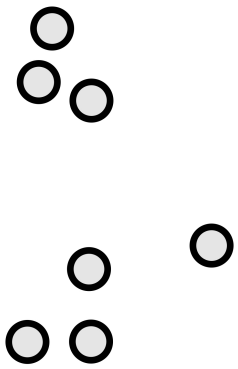


Figure 1: Toy dataset used to illustrate disconnected neighborhood graphs.

If Isomap is applied using a very small neighborhood size (e.g. $\rho = 1$), the resulting neighborhood graph contains multiple disconnected components:

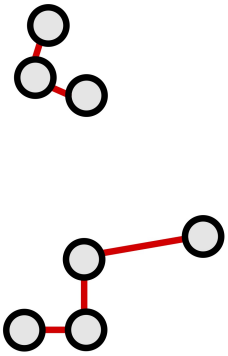


Figure 2: Neighborhood graph for $\rho = 1$.

As shown in the figure, the local neighborhoods do not connect to each other, causing the graph to fragment into two disconnected components.

Question 3E.2.2

Since Isomap approximates geodesic distances using shortest paths in the neighborhood graph G, a disconnected graph makes these distances undefined between points belonging to different connected components. Consequently, geodesic distances cannot be computed for all pairs of points, and the subsequent multidimensional scaling (MDS) step cannot produce a single meaningful embedding of the entire dataset. This is undesirable because Isomap then fails to capture and preserve the global manifold structure.

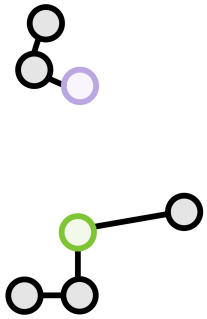


Figure 3: $\rho = 1$: the geodesic distance between the two highlighted points is undefined.

Question 3E.2.3

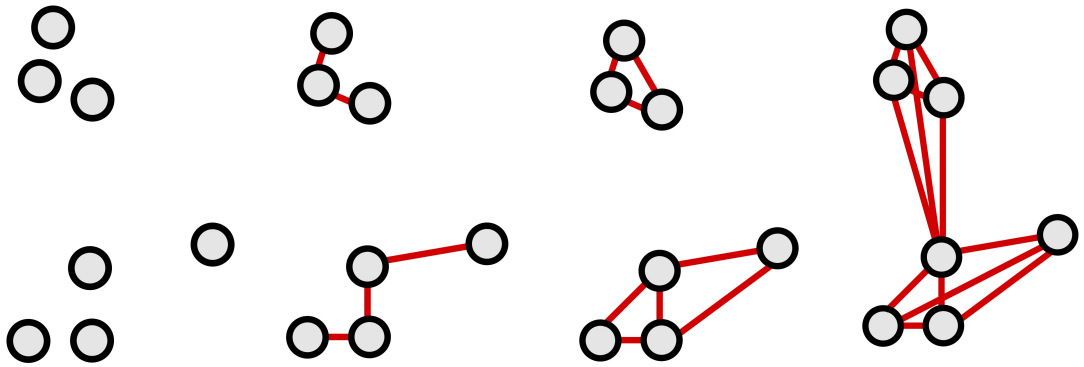
A simple and effective heuristic to avoid obtaining a disconnected neighborhood graph in Isomap is to adopt an incremental strategy for selecting the neighborhood size ρ.

The idea is to start with a small value of ρ (e.g. $\rho = 1$) and gradually increase it, for instance in steps of one, until the neighborhood graph becomes connected. At each

iteration, the geodesic distance matrix is computed using shortest paths in the graph. If the graph is disconnected, some pairwise geodesic distances will be undefined; in practice, these can be represented as ∞ or NaN values. The procedure is repeated until all pairwise geodesic distances are defined, indicating that the graph is connected.

The intuition behind this heuristic is to use the smallest neighborhood size that guarantees global connectivity, thereby preserving the local geometry of the manifold while still allowing information to propagate across the entire dataset. This approach is expected to work well in practice because it adapts the neighborhood size to the data density: sparse regions are connected only when necessary, while overly large neighborhoods that would introduce short-circuiting edges are avoided.

In the following, the heuristic is applied to the previous toy dataset.



(a) $\rho = 0$ (b) $\rho = 1$ (c) $\rho = 2$ (d) $\rho = 3$

Figure 4: Neighborhood graph as a function of the neighborhood size ρ. For small values of ρ the graph is disconnected; increasing ρ progressively merges local components until the graph becomes connected.

3E.3 PCA vs. Johnson-Lindenstrauss random projections

Question 3E.3.1

- **Projection error**
  In PCA the criterion to be optimized is the mean squared reconstruction error, defined as

$$E_{A_{\text{PCA}}} = \mathbb{E}_y \left[\|y - \text{dec}(\text{cod}(y))\|_2^2 \right] = \mathbb{E}_y \left[\|y - A_{\text{PCA}}^T A_{\text{PCA}}\, y\|_2^2 \right]$$

  The main idea behind PCA is to minimize, on average over the data distribution, the squared distance between each data point and its reconstruction in the reduced subspace. This makes PCA optimal in terms of expected reconstruction error, but it does not provide explicit guarantees on preserving pairwise distances between points. According to this criterion, it follows that

$$A_{\text{PCA}} = U_k^T, \tag{1}$$

where U_k denotes the matrix formed by the first k columns of U obtained from the singular value decomposition $Y = U\Sigma V^T$.

On the other hand, Johnson–Lindenstrauss random projections are explicitly designed to preserve pairwise distances between points after projection, i.e.

$$\|y_i - y_j\|_2 \approx \|f(y_i) - f(y_j)\|_2 = \|A_{\mathrm{JL}}\, y_i - A_{\mathrm{JL}}\, y_j\|_2\,, \quad \forall y_i, y_j \in Y.$$

According to the Johnson–Lindenstrauss theorem, this property holds with high probability for all pairs of points, independently of the data distribution. From this result, the projection matrix can be expressed as

$$A_{\mathrm{JL}} = \sqrt{\frac{d}{k}}\, Z \tag{2}$$

where $Z \in \mathbb{R}^{k\times d}$ is the matrix whose rows are k independent random vectors drawn uniformly from the unit sphere in $\mathbb{R}^d$.

Unlike PCA, JL projections do not minimize a reconstruction error, but instead control the distortion of pairwise distances.

- **Computational efficiency**
  From a computational perspective, PCA is more expensive to compute, since it requires performing a singular value decomposition in order to determine the projection matrix A_{PCA}. This step involves a non-negligible computational cost and must be repeated whenever the data distribution changes.

  In contrast, Johnson–Lindenstrauss random projections do not require any training phase: the projection matrix A_{JL} is obtained by directly drawing random vectors from the uniform distribution.

  As a result, JL projections are computationally cheaper to construct and can be applied immediately, independently of the data.

- **Target usecases**
  PCA is preferred when reconstruction quality and data adaptivity are required, whereas JL projections are more appropriate when distance preservation and scalability are the primary concerns.

3E.4 Spectral graph analysis

Question 3.4.1

Considering a d-regular graph G=(V,E) and A its adjacency matrix with the normalized Laplacian of G as $L = I - \frac{1}{d}A$. We want to prove that for any vector $x \in R^{|V|}$ it is

$$x^T L x = \frac{1}{d} \sum_{(u,v)\in E} (x_u - x_v)^2$$

We start from the right-hand side and work our way to $x^T Lx$

$$\frac{1}{d}\sum_{(u,v)\in E}(x_u - x_v)^2 = \frac{1}{d}\left(\sum_{(u,v)\in E}\left(x_u^2 + x_v^2\right) - 2\sum_{(u,v)\in E} x_u x_v\right)$$

Since the graph is undirected, the sum $\sum_{(u,v)\in E}\left(x_u^2 + x_v^2\right)$ ranges over unordered edges, each appearing exactly once. For a fixed vertex $w \in V$, the term x_w^2 appears once for every edge incident to w. Because the graph is d-regular, vertex w is incident to exactly d edges. Hence x_w^2 appears exactly d times in the sum, and therefore

$$\sum_{(u,v)\in E}(x_u^2 + x_v^2) = d\sum_{w\in V} x_w^2.$$

We can replace it in the proof.

$$\begin{aligned}
\frac{1}{d}\sum_{(u,v)\in E}(x_u - x_v)^2 &= \frac{1}{d}\left(d\sum_{(w)\in E} x_w^2 - 2\sum_{(u,v)\in E} x_u x_v\right) \\
&= \sum_{(w)\in E} x_w^2 - \frac{2}{d}\sum_{(u,v)\in E} x_u x_v \\
&= x^T x - \frac{2}{d}\left(\frac{1}{2}\sum_{u,v} A_{uv} x_u x_v\right) \\
&= x^T x - \frac{1}{d}x^T Ax \\
&= x^T\left(I - \frac{1}{d}A\right)x \\
&= x^T Lx
\end{aligned}$$

Question 3.4.2

We want to prove that the normalized Laplacian ($L = I - \frac{1}{d}A$) is a positive semidefinite matrix.
A matrix is positive semidefine if $x^T Lx \geq 0 \quad \forall x \in R^{|V|}$
We know from the theory and from the previous exercise that:

$$x^T Lx = \frac{1}{d}\sum_{(u,v)\in E}(x_u - x_v)^2$$

The expression $x^T Lx$ is defined as a sum of squared values, it means that its final result will be always greater or equal to 0 by definition.

Question 3E.4.3

In linear algebra, the trivial vector is the zero vector $\mathbf{0}$. In this optimization problem, however, there is also an uninformative family of solutions given by any constant vector $x = c\,\mathbf{1}$. Indeed, in this case all vertices receive the same coordinate, hence the embedding collapses to a single point and carries no information about the graph geometry.

To interpret the 1D embedding induced by a non-trivial minimizer x_*, we use Equation (4):

$$x^\top L x = \frac{1}{d} \sum_{(u,v)\in E} (x_u - x_v)^2.$$

Minimizing $x^\top L x$ means making $(x_u - x_v)^2$ small for every edge (u, v), that is, it encourages adjacent vertices to be mapped to nearby points on the real line. In other words, the entries of x_* can be used as 1D coordinates, assigning to each vertex v the real value $(x_*)_v$.

Appendix - Code

See the following pages.